jupyter potato-disease-classification-model Last Checkpoint: 14 days ago

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python 3 (ipykernel)

Potato Disease Classification

Dataset credits: https://www.kaggle.com/arjuntejaswi/plant-village

Import all the Dependencies

[3]:
```
import tensorflow as tf
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
from IPython.display import HTML
```

Set all the Constants

[4]:
```
BATCH_SIZE = 32
IMAGE_SIZE = 256
CHANNELS=3
EPOCHS=50
```

Import data into tensorflow dataset object

We will use image_dataset_from_directory api to load all images in tensorflow dataset:
https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory

[5]:
```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "PlantVillage",
    seed=123,
    shuffle=True,
    image_size=(IMAGE_SIZE,IMAGE_SIZE),
    batch_size=BATCH_SIZE
)
```

```
Found 2152 files belonging to 3 classes.
```

Watch below video on tensorflow input pipeline first if you don't know about tensorflow datasets

[6]:
```
class_names = dataset.class_names
class_names
```

[6]:
```
['Potato___Early_blight', 'Potato___Late_blight', 'Potato___healthy']
```

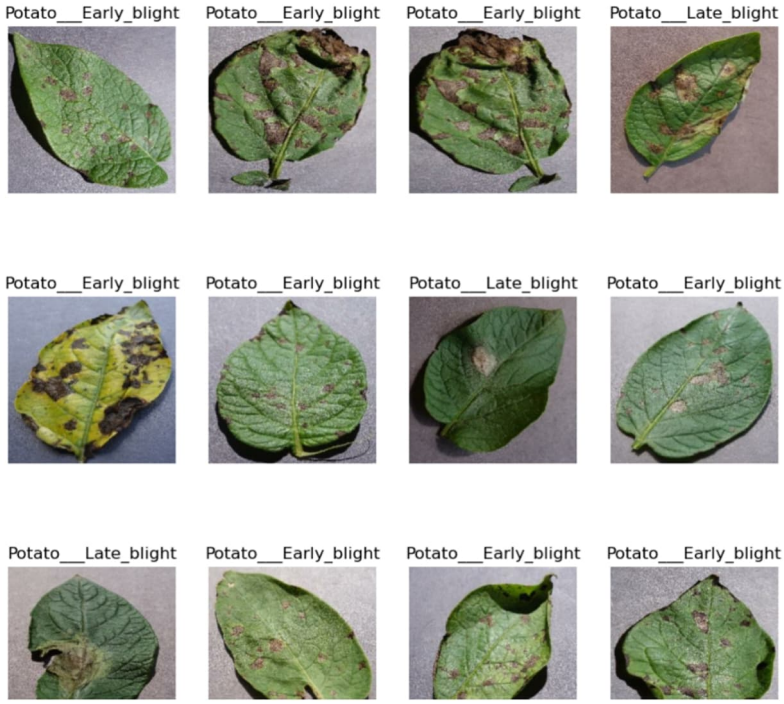
[7]:
```
for image_batch, labels_batch in dataset.take(1):
    print(image_batch.shape)
    print(labels_batch.numpy())
```

```
(32, 256, 256, 3)
[1 1 1 0 0 0 0 0 1 1 1 1 0 1 0 1 1 1 0 1 0 1 0 0 1 0 0 1 1 2 0 0]
```

As you can see above, each element in the dataset is a tuple. First element is a batch of 32 elements of images. Second element is a batch of 32 elements of class labels

Visualize some of the images from our dataset

[8]:
```
plt.figure(figsize=(10, 10))
for image_batch, labels_batch in dataset.take(1):
    for i in range(12):
        ax = plt.subplot(3, 4, i + 1)
        plt.imshow(image_batch[i].numpy().astype("uint8"))
        plt.title(class_names[labels_batch[i]])
        plt.axis("off")
```



Function to Split Dataset

Dataset should be bifurcated into 3 subsets, namely:

1. Training: Dataset to be used while training
2. Validation: Dataset to be tested against while training
3. Test: Dataset to be tested against after we trained a model

[9]:
```
len(dataset)
```

[9]: 68

[10]:
```
train_size = 0.8
len(dataset)*train_size
```

[10]: 54.400000000000006

[11]:
```
train_ds = dataset.take(54)
len(train_ds)
```

[11]: 54

[12]:
```
test_ds = dataset.skip(54)
len(test_ds)
```

[12]: 14

[13]:
```
val_size=0.1
len(dataset)*val_size
```

[13]: 6.800000000000001

[14]:
```
val_ds = test_ds.take(6)
len(val_ds)
```

[14]: 6

[15]:
```
test_ds = test_ds.skip(6)
len(test_ds)
```

[15]: 8

[16]:
```
def get_dataset_partitions_tf(ds, train_split=0.8, val_split=0.1, test_split=0.1, shuffle=True, shuffle_size=10000):
    assert (train_split + test_split + val_split) == 1

    ds_size = len(ds)

    if shuffle:
        ds = ds.shuffle(shuffle_size, seed=12)

    train_size = int(train_split * ds_size)
    val_size = int(val_split * ds_size)

    train_ds = ds.take(train_size)
    val_ds = ds.skip(train_size).take(val_size)
    test_ds = ds.skip(train_size).skip(val_size)

    return train_ds, val_ds, test_ds
```

[17]:
```
train_ds, val_ds, test_ds = get_dataset_partitions_tf(dataset)
```

[18]:
```
len(train_ds)
```

[18]: 54

[19]:
```
len(val_ds)
```

[19]: 6

[20]:
```
len(test_ds)
```

[20]: 8

Cache, Shuffle, and Prefetch the Dataset

[21]:
```
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds = val_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
test_ds = test_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
```

Building the Model

Creating a Layer for Resizing and Normalization

Before we feed our images to network, we should be resizing it to the desired size. Moreover, to improve model performance, we should normalize the image pixel value (keeping them in range 0 and 1 by dividing by 256). This should happen while training as well as inference. Hence we can add that as a layer in our Sequential Model.

You might be thinking why do we need to resize (256,256) image to again (256,256). You are right we don't need to but this will be useful when we are done with the training and start using the model for predictions. At that time somone can supply an image that is not (256,256) and this layer will resize it

[22]:
```
resize_and_rescale = tf.keras.Sequential([
    layers.Resizing(IMAGE_SIZE, IMAGE_SIZE),
    layers.Rescaling(1./255),
])
```

Data Augmentation

Data Augmentation is needed when we have less data, this boosts the accuracy of our model by augmenting the data.

[23]:
```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
```

```
])
```

Applying Data Augmentation to Train Dataset

[24]:
```
train_ds = train_ds.map(
    lambda x, y: (data_augmentation(x, training=True), y)
).prefetch(buffer_size=tf.data.AUTOTUNE)
```

Model Architecture

We use a CNN coupled with a Softmax activation in the output layer. We also add the initial layers for resizing, normalization and Data Augmentation.

[25]:
```
input_shape = (BATCH_SIZE, IMAGE_SIZE, IMAGE_SIZE, CHANNELS)
n_classes = 3

model = models.Sequential([
    resize_and_rescale,
    layers.Conv2D(32, kernel_size = (3,3), activation='relu', input_shape=input_shape),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(n_classes, activation='softmax'),
])

model.build(input_shape=input_shape)
```

```
C:\Users\hp5cd\AppData\Roaming\Python\Python312\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[26]:
```
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(32, 256, 256, 3)	0
conv2d (Conv2D)	(32, 254, 254, 32)	896
max_pooling2d (MaxPooling2D)	(32, 127, 127, 32)	0
conv2d_1 (Conv2D)	(32, 125, 125, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(32, 62, 62, 64)	0
conv2d_2 (Conv2D)	(32, 60, 60, 64)	36,928
max_pooling2d_2 (MaxPooling2D)	(32, 30, 30, 64)	0
conv2d_3 (Conv2D)	(32, 28, 28, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(32, 14, 14, 64)	0
conv2d_4 (Conv2D)	(32, 12, 12, 64)	36,928
max_pooling2d_4 (MaxPooling2D)	(32, 6, 6, 64)	0
conv2d_5 (Conv2D)	(32, 4, 4, 64)	36,928
max_pooling2d_5 (MaxPooling2D)	(32, 2, 2, 64)	0
flatten (Flatten)	(32, 256)	0
dense (Dense)	(32, 64)	16,448
dense_1 (Dense)	(32, 3)	195

Total params: 183,747 (717.76 KB)
Trainable params: 183,747 (717.76 KB)
Non-trainable params: 0 (0.00 B)

Compiling the Model

We use `adam` Optimizer, `SparseCategoricalCrossentropy` for losses, `accuracy` as a metric

[27]:
```
model.compile(optimizer='adam', loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True, reduction='sum_over_batch_size'))
```

[28]:
```
history = model.fit(
    train_ds,
    batch_size=BATCH_SIZE,
    validation_data=val_ds,
    verbose=1,
    epochs=50,
)
```

```
Epoch 1/50
C:\Users\hp5cd\AppData\Roaming\Python\Python312\site-packages\keras\src\backend\tensorflow\nn.py:708: UserWarning: "`sparse_categorical_crossentropy` received `from_logits=True`, but the `output` argument was produced by a Softmax activation and thus does not represent logits. Was this intended?
  output, from_logits = _get_logits(
54/54 ━━━━━━━━━━━━━━━━━━━━ 49s 812ms/step - loss: 0.9119 - val_loss: 0.6487
Epoch 2/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 47s 866ms/step - loss: 0.6009 - val_loss: 0.4400
Epoch 3/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 42s 771ms/step - loss: 0.3599 - val_loss: 0.3457
Epoch 4/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 801ms/step - loss: 0.3129 - val_loss: 0.3819
Epoch 5/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 82s 806ms/step - loss: 0.2240 - val_loss: 0.2674
Epoch 6/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 41s 764ms/step - loss: 0.2086 - val_loss: 0.2098
Epoch 7/50
```

```
Epoch 7/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 40s 742ms/step - loss: 0.2165 - val_loss: 0.2029
Epoch 8/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 40s 742ms/step - loss: 0.2154 - val_loss: 0.2145
Epoch 9/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 815ms/step - loss: 0.1682 - val_loss: 0.1542
Epoch 10/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 809ms/step - loss: 0.1681 - val_loss: 0.1597
Epoch 11/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 794ms/step - loss: 0.1679 - val_loss: 0.4826
Epoch 12/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 48s 886ms/step - loss: 0.1718 - val_loss: 0.1214
Epoch 13/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 795ms/step - loss: 0.1291 - val_loss: 0.1223
Epoch 14/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 814ms/step - loss: 0.0924 - val_loss: 0.1054
Epoch 15/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 803ms/step - loss: 0.0908 - val_loss: 0.0802
Epoch 16/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 46s 842ms/step - loss: 0.1043 - val_loss: 0.1820
Epoch 17/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 42s 777ms/step - loss: 0.1100 - val_loss: 0.0798
Epoch 18/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 45s 825ms/step - loss: 0.0834 - val_loss: 0.0971
Epoch 19/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 786ms/step - loss: 0.0988 - val_loss: 0.0782
Epoch 20/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 802ms/step - loss: 0.1301 - val_loss: 0.1124
Epoch 21/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 42s 783ms/step - loss: 0.0914 - val_loss: 0.0565
Epoch 22/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 788ms/step - loss: 0.1123 - val_loss: 0.1613
Epoch 23/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 52s 964ms/step - loss: 0.0744 - val_loss: 0.0823
Epoch 24/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 51s 937ms/step - loss: 0.0904 - val_loss: 0.1668
Epoch 25/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 49s 911ms/step - loss: 0.0823 - val_loss: 0.0322
Epoch 26/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 48s 887ms/step - loss: 0.0657 - val_loss: 0.4105
Epoch 27/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 816ms/step - loss: 0.1220 - val_loss: 0.2165
Epoch 28/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 48s 876ms/step - loss: 0.0642 - val_loss: 0.1283
Epoch 29/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 42s 777ms/step - loss: 0.0581 - val_loss: 0.0286
Epoch 30/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 794ms/step - loss: 0.0922 - val_loss: 0.0328
Epoch 31/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 791ms/step - loss: 0.0592 - val_loss: 0.1033
Epoch 32/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 44s 814ms/step - loss: 0.0273 - val_loss: 0.0486
Epoch 33/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 43s 801ms/step - loss: 0.0580 - val_loss: 0.0371
Epoch 34/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 45s 826ms/step - loss: 0.0467 - val_loss: 0.1591
Epoch 35/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 49s 910ms/step - loss: 0.0413 - val_loss: 0.0305
Epoch 36/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 52s 964ms/step - loss: 0.0663 - val_loss: 0.1789
Epoch 37/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 49s 897ms/step - loss: 0.0730 - val_loss: 0.1352
Epoch 38/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 48s 894ms/step - loss: 0.0412 - val_loss: 0.0792
Epoch 39/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 48s 891ms/step - loss: 0.0381 - val_loss: 0.0992
Epoch 40/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 49s 911ms/step - loss: 0.0413 - val_loss: 0.4739
Epoch 41/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 51s 943ms/step - loss: 0.0644 - val_loss: 0.0113
Epoch 42/50
 5/54 ━━━━━━━━━━━━━━━━━━━━ 42s 868ms/step - loss: 0.0630
```

[ ]:
```
scores = model.evaluate(test_ds)
```

You can see above that we get 100.00% accuracy for our test dataset. This is considered to be a pretty good accuracy

[ ]:
```
scores
```

Scores is just a list containing loss and accuracy value

Plotting the Accuracy and Loss Curves

[ ]:
```
history
```

You can read documentation on history object here: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/History

[ ]:
```
history.params
```

[ ]:
```
history.history.keys()
```

loss, accuracy, val loss etc are a python list containing values of loss, accuracy etc at the end of each epoch

[ ]:
```
type(history.history['loss'])
```

[ ]:
```
len(history.history['loss'])
```

```
[ ]: len(history.history['loss'])
```

```
[ ]: history.history['loss'][:5] # show loss for first 5 epochs
```

```
[ ]: acc = history.history['accuracy']
     val_acc = history.history['val_accuracy']

     loss = history.history['loss']
     val_loss = history.history['val_loss']
```

```
[ ]: plt.figure(figsize=(8, 8))
     plt.subplot(1, 2, 1)
     plt.plot(range(EPOCHS), acc, label='Training Accuracy')
     plt.plot(range(EPOCHS), val_acc, label='Validation Accuracy')
     plt.legend(loc='lower right')
     plt.title('Training and Validation Accuracy')

     plt.subplot(1, 2, 2)
     plt.plot(range(EPOCHS), loss, label='Training Loss')
     plt.plot(range(EPOCHS), val_loss, label='Validation Loss')
     plt.legend(loc='upper right')
     plt.title('Training and Validation Loss')
     plt.show()
```

Run prediction on a sample image

```
[ ]: import numpy as np
     for images_batch, labels_batch in test_ds.take(1):

         first_image = images_batch[0].numpy().astype('uint8')
         first_label = labels_batch[0].numpy()

         print("first image to predict")
         plt.imshow(first_image)
         print("actual label:",class_names[first_label])

         batch_prediction = model.predict(images_batch)
         print("predicted label:",class_names[np.argmax(batch_prediction[0])])
```

Write a function for inference

```
[ ]: def predict(model, img):
         img_array = tf.keras.preprocessing.image.img_to_array(images[i].numpy())
         img_array = tf.expand_dims(img_array, 0)

         predictions = model.predict(img_array)

         predicted_class = class_names[np.argmax(predictions[0])]
         confidence = round(100 * (np.max(predictions[0])), 2)
         return predicted_class, confidence
```

Now run inference on few sample images

```
[ ]: plt.figure(figsize=(15, 15))
     for images, labels in test_ds.take(1):
         for i in range(9):
             ax = plt.subplot(3, 3, i + 1)
             plt.imshow(images[i].numpy().astype("uint8"))

             predicted_class, confidence = predict(model, images[i].numpy())
             actual_class = class_names[labels[i]]

             plt.title(f"Actual: {actual_class},\n Predicted: {predicted_class}.\n Confidence: {confidence}%")

             plt.axis("off")
```

Saving the Model

We append the model to the list of models as a new version

```
[ ]: import os
     model_version=max([int(i) for i in os.listdir("../models") + [0]])+1
     model.save(f"../models/{model_version}.keras")
```

```
[ ]: model.save("../potatoes.h5")
```

```
[ ]:
```